

MetroBlue ML Service — Intern Feedback & Gap Closure Plan

Project: metroblue-leadscoring-api | Audit date: 2026-04-08

1. Executive Summary

The repository delivers roughly one-third of the agreed three-week scope. Intern A (Musrat) produced a working lead-scoring ML pipeline but it lives in isolation and is never invoked by the production API. Intern B (Gabino) delivered no code at all — the entire revenue forecasting workstream is missing. Intern C (Thomas) built the FastAPI scaffold and a static HTML dashboard, but every business endpoint returns hardcoded mock data, the database layer is wired to SQLite instead of MySQL, and only one ORM model exists. The service cannot currently serve a real prediction end-to-end.

Approximate completion by intern:

- Intern A — Lead Scoring: ~78% (16 done, 4 partial, 3 missing out of 23 items)
- Intern B — Revenue Forecasting: 0% (0 of 13 items)
- Intern C — FastAPI / Dashboard: ~21% fully implemented (2 done, 9 partial, 8 missing out of 19)
- Tech stack & packaging: ~55% (no Dockerfile, no root requirements.txt, no .env.example)

2. Intern A — Musrat (Lead Scoring)

What went well

- Clean separation across `db_extract.py`, `preprocess.py`, `train.py`, `predict.py` — easy to follow.
- Solid feature engineering: nearly every feature in the spec is present and correct.
- Both Logistic Regression and Random Forest are trained, evaluated with stratified K-fold, and the better model is auto-selected by F1.
- `class_weight='balanced'` handles imbalance pragmatically; feature-importance chart is generated.
- `score_lead()` returns the right shape and a usable Cold/Warm/Hot label band.

Gaps to close

- Module is never imported by `main.py` — your work is invisible to the API. Move `Musrat_Task/` to a proper package named `lead_scoring/` at the repo root and expose `score_lead` from `lead_scoring/__init__.py`.

- DB credentials are hardcoded in `db_extract.py`. Read them from environment variables (`DATABASE_URL`) so the same connection layer is shared with the API.
- EDA artifacts (stage distribution, source breakdown, conversion rates) are missing. Add a `notebooks/eda_leads.ipynb` or `eda_report.md` with the charts and key numbers.
- Unit tests referenced in your `readme.md` (`tests/test_predict.py`) do not exist. Create at least 3 `pytest` cases: a hot lead, a cold lead, and a missing-fields lead.
- `top_factors` in `predict.py` is hand-coded with if-statements. Derive it from the model: for Random Forest use `feature_importances_` * the lead's one-hot row to get the top 3 contributing features for that specific lead.
- No scaler/encoder is persisted with `joblib` — only the model and column list. If you ever swap to Logistic Regression as the winner, predictions will silently drift. Persist a `sklearn Pipeline` (preprocessor + classifier) so the whole transform is one artifact.
- `gender` is one-hot encoded with 3 categories instead of binary as the spec requested. Either map to {Male: 1, Female: 0, Unknown: -1} or document the deviation.
- `readme.md` is truncated mid sentence. Finish it: include training command, expected inputs/outputs, model metrics on the latest run, and how to retrain.
- Random Forest hyperparameters are hardcoded. Add a small `GridSearchCV` (`n_estimators` in [100, 200, 400], `max_depth` in [5, 10, None]) so 'tuning' is real.

3. Intern B — Gabino (Revenue Forecasting)

No code has been committed for this workstream. The endpoints in `main.py` that look like forecasting (`/api/revenue/forecast`, `/api/revenue/overdue`, `/api/clients/risk`) return hardcoded ABC Corp / XYZ Ltd rows written by Intern C as placeholders. The entire 3-week scope is outstanding.

Required deliverables to close the gap

Week 1 — Data foundation:

- Create a `revenue_forecasting/` package with `db_extract.py` that pulls from `payments`, `sales_payment_records`, and `sales_payment_installments` using the shared `DATABASE_URL`.
- Build a monthly aggregation: total revenue per month for the last 24 months, broken down by `payment_type` and category.
- Compute installment behavior metrics per client: % paid on-time, average days late, completion rate. Save as a reusable `DataFrame` builder.
- Produce an EDA artifact (notebook or markdown) showing monthly revenue trend, top categories, and the on-time-rate distribution.

Week 2 — Modeling:

- Implement `scheduled_revenue_baseline()`: for each future month, sum unpaid installments whose `due_date` falls in that month.

- Implement a historical trend model. Start with statsmodels Linear Regression on month index; if time permits, add ARIMA or Holt-Winters from statsmodels.
- Combine the two: $\text{predicted} = \text{scheduled} * \text{payment_reliability_factor} + \text{trend_residual}$. Document the formula in the module docstring.
- Backtest on the last 2-3 months (hold-out) and report MAE and MAPE in a backtest_report.md.
- Add per-category forecasting (recurring vs one-time) so total can be sanity-checked against the sum.

Week 3 — Serving:

- Expose forecast_revenue(months_ahead=3) returning [{month, predicted, scheduled, confidence}] where confidence is the 80% prediction interval half-width from the trend model.
- Expose get_overdue_installments() returning [{client_name, amount, days_overdue, installment_number}] from sales_payment_installments WHERE due_date < CURDATE() AND paid_at IS NULL.
- Expose score_client_risk() returning [{name, risk_score, on_time_rate, avg_delay_days}]. $\text{risk_score} = 1 - \text{on_time_rate}$ adjusted for avg delay.
- Persist any fitted models with joblib in models/revenue_model.pkl. Add a retrain CLI mirroring Intern A's pattern.
- Add pytest unit tests with a small fixture DataFrame; do not require a live DB to run them.
- Write a revenue_forecasting/README.md that explains the approach, formulas, and how to retrain.

4. Intern C — Thomas (FastAPI Service & Dashboard)

What went well

- FastAPI app boots, has a global exception handler, CORS, structured success/error envelope, and a logging module.
- API key dependency exists and is wired into every protected endpoint.
- Dashboard pages exist for the three required views and use Chart.js as specified.

Critical gaps

- EVERY business endpoint is a stub. /api/leads/score returns 0.8 if source is 'google' else 0.4. /api/leads/scores assigns score = 0.5 with a 'replace with real logic' comment. The three revenue endpoints return hardcoded ABC Corp / XYZ Ltd rows. /api/models/retrain spawns retrain.py at the repo root, which is itself a print + time.sleep stub. None of Intern A's real ML code is imported.
- database.py is configured for SQLite (check_same_thread=False). The spec requires MySQL. Switch to mysql+pymysql:// with a connection pool and pool_pre_ping=True.

- `models.py` only declares `Lead` with 4 columns (`id`, `name`, `source`, `course_service`). The spec requires read-only ORM models for `leads`, `payments`, `sales_payment_records`, `sales_payment_installments`, `clients`, `referrals`. Build them all and mark them read-only by never exposing a write session in the API layer.
- `Base.metadata.create_all(bind=engine)` is called at startup. This will try to CREATE the `leads` table on the production MySQL DB — remove it. The DB schema is owned by the main `MetroBlue` app.
- `/api/seed` exists and inserts data into the `leads` table. Delete it before any deployment.
- Dashboard ↔ API contract is broken: `forecast.html` reads `result.data.results`, but the API returns `result.data.forecast`. The chart will always render empty. Fix the JS or change the API key.
- API key is hardcoded in dashboard JS files — and the values do not even match each other ('API KEY Private' in `app.js`, 'secret123' in `forecast.html` and `overdue.html`). Move to a config endpoint or inject at build time, and never commit a real key.
- The Lead Scoreboard has a stage `<select>` but no event listener. Add the filter handler. Add a click handler on the Score column header to sort.
- `forecast.html` has no historical data series and no confidence band. Plot two datasets (history + forecast) and a shaded area for the prediction interval.
- `overdue.html` does not display client risk at all — only overdue rows. Either add a second table or call `/api/clients/risk` and merge.
- No Dockerfile. Add one based on `python:3.11-slim`, install requirements, copy app, run `uvicorn`.
- No root requirements.txt or `pyproject.toml`. `fastapi`, `uvicorn`, `sqlalchemy`, `pymysql`, `python-dotenv`, `scikit-learn`, `pandas`, `joblib`, `statsmodels`, `pytest`, `httpx` must all be declared.
- `integration.md` is 15 lines and only documents one endpoint. Document every endpoint with request shape, response shape, and a `fetch()` example.
- No `.env.example`. Add one listing `DATABASE_URL`, `API_KEY`, `MODEL_DIR`, `LOG_LEVEL`.

5. Code Quality Recommendations (all interns)

- Adopt a single source-tree layout: `app/` for the FastAPI service, `lead_scoring/` for Intern A, `revenue_forecasting/` for Intern B, `dashboard/` for the UI, `tests/` at the root. Delete `Musrat_Task/` once the move is complete.
- Add `ruff` + `black` + `isort` and run them in a pre-commit hook. Code style across the three folders is currently inconsistent.
- Add type hints to all public functions. `score_lead`, `forecast_revenue`, and every endpoint should have annotated arguments and return types so FastAPI generates an accurate OpenAPI schema.
- Pull magic numbers out of code (score thresholds 0.30/0.60, `top_locations=10`, `RF_n_estimators=200`) into a config.py module.

- Replace `print()` in scripts with the logger from `core/logging.py` so output is consistent in production logs.
- Centralize the DB engine in `app/db.py` and import it everywhere. Today `database.py` and `Musrat_Task/db_extract.py` each create their own engine with different drivers.
- Use FastAPI dependency injection for the DB session (yield pattern) instead of `SessionLocal()` / `db.close()` in every handler. The current code leaks sessions on exceptions.
- Use Pydantic `response_model` on every endpoint. Today only the request is validated; responses are loose dicts and the dashboard already has bugs because of it.
- Wrap model loading in a module-level lazy singleton so `joblib.load` is not called on every request to `/api/leads/score`.
- Add a Makefile or invoke tasks for: `make install`, `make test`, `make lint`, `make train`, `make run`.
- Set up CI (GitHub Actions): `install`, `lint`, `run pytest`. This would have caught the missing tests immediately.
- Aim for $\geq 60\%$ test coverage before sign-off. At minimum: 1 test per endpoint hitting a SQLite test fixture; 1 test per model serializer; 1 test per feature transform.

6. Security Recommendations

- API key is stored in a single env var and compared with `==`. Use `hmac.compare_digest` to avoid timing attacks, and support key rotation by allowing a comma-separated list of valid keys.
- Move authentication to a real FastAPI middleware or `APIKeyHeader` security scheme so it shows up in `/docs` and is enforced uniformly.
- `CORS` is `allow_origins=['*']` with `allow_credentials=True`. This is invalid per the `CORS` spec and dangerous. Restrict to the dashboard origin(s) and remove the wildcard if credentials are needed.
- Hardcoded API keys in `dashboard/app.js`, `dashboard/forecast.html`, `dashboard/overdue.html` must be removed before any commit. Treat the current values as compromised and rotate.
- Hardcoded MySQL credentials in `Musrat_Task/db_extract.py` (root user, empty password) must be removed. Use the shared `DATABASE_URL`.
- Database connection must be a read-only MySQL user. Create a `metroblue_ml_ro` account with `SELECT`-only grants on the six required tables. This protects against the still-present `/api/seed` endpoint and any future bug.
- Delete `/api/seed` and the `Base.metadata.create_all()` call. Both can mutate the production schema.
- Global exception handler returns 'Internal server error' which is good, but the underlying `logger.error` logs the raw exception. Make sure logs do not capture full SQL query strings or PII (lead names, phones, notes).

- `/api/models/retrain` runs `subprocess.Popen(['python3', script_path])` with no auth scoping beyond the API key, no concurrency lock, and no audit log. Add: a single-flight lock file, an admin-only key tier, and an audit entry with caller IP and timestamp.
- `/api/models/retrain` shells out using the current working directory (`os.getcwd()`) which is attacker-influencable in some deploy setups. Use an absolute path resolved from `__file__`.
- Add input length and value caps on `/api/leads/score` (e.g. `notes ≤ 4000` chars, `source/course_service ≤ 100` chars) to prevent abuse and runaway feature transforms.
- Pin all dependency versions in `requirements.txt` and run `pip-audit` (or `Dependabot`) on a schedule.
- Add rate limiting (`slowapi` or an upstream `nginx` limit) on `/api/leads/score` and `/api/models/retrain`.
- Make sure `.env`, `*.pkl` model files, and any DB dumps are listed in `.gitignore`. Currently only `.gitignore` exists at the root — verify the contents.
- Set `Uvicorn` behind a reverse proxy (`nginx/Caddy`) with TLS. Do not expose port 8000 directly.

7. Prioritized Action List (close-out order)

Do these in order. Each step unblocks the next.

- P0 — Intern C: switch `database.py` to MySQL with `pool_pre_ping`; remove `create_all()` and `/api/seed`; add the missing ORM models as read-only.
- P0 — Intern C: wire `/api/leads/score` and `/api/leads/scores` to Intern A's `score_lead`. Delete the stub branches.
- P0 — Intern A: restructure `Musrat_Task/` into a `lead_scoring/` package and remove hardcoded DB credentials.
- P0 — Rotate the leaked dashboard API keys and remove them from the JS files.
- P1 — Intern B: deliver the `revenue_forecasting/` module end-to-end (Week 1 → Week 3 above).
- P1 — Intern C: replace the three revenue endpoints with calls into `revenue_forecasting/`.
- P1 — Intern C: write a real Dockerfile and root `requirements.txt`; add `.env.example`.
- P2 — All: add `pytest` tests, `ruff/black`, and a GitHub Actions CI pipeline.
- P2 — Intern C: fix dashboard contracts (`forecast.html` field names, scoreboard sort/filter handlers, `overdue+risk` merge).
- P3 — All: finish documentation — module READMEs, `integration.md` endpoint catalog, and a top-level architecture diagram.

8. Definition of Done

- A fresh clone, `.env` populated, `docker build && docker run`, and `curl -H 'x-api-key: ...' http://localhost:8000/api/leads/score` returns a score from the real model.
- All six endpoints in the spec return real data with the correct JSON shape.

- pytest passes with at least one test per module.
- Dashboard loads all three pages, charts render with live data, no console errors.
- No hardcoded credentials anywhere in the repo.
- README at the root explains setup, training, running, and deployment in under 5 minutes of reading.